You give it a module containing one or more refinements, and it marks the refined objects to say "for the rest of the scope of this method, when you make a call to an instance of Object, first check to see if the method can be found in the refinement. If so, invoke it, otherwise invoke the original."

The basic scoping rule is that a refinement is activated in a source file by calling using. If using is called inside a module or class, the refinement is in effect for the duration of that definition.

You can also invoke using at the top level of a file, in which case for the rest of that source file, the methods that are defined in that refinement are active inside any classes or methods yet to be defined in the source file:

metaprogramming/use_in_file.rb

```
require_relative "vanity_puts"
using VanityPuts
puts "Hello", "world"
```

produces:

```
Dave says: Hello
Dave says: world
```

Let's step it up a notch. We'll define three source files. Here's one that contains a refinement definition:

metaprogramming/ref1/vanity_refinement.rb

```
module VanityPuts
  refine Object do
    private def puts(*args)
      args.each do |arg|
        Kernel.puts("Dave says: #{arg}")
      end
    end
  end
end
```

And here's a file that uses this refinement:

metaprogramming/ref1/file_using_refinement.rb

```
class VanityShouter
  using VanityPuts

  def shout(msg)
    puts(msg.upcase)
  end
end

VanityShouter.new.shout("I'm here!")
```

Finally, let's run them from a third file:

metaprogramming/ref1/main_program.rb

```
require_relative "vanity_refinement"

puts "About to require file using refinement"
require_relative "file_using_refinement"
puts "Back from require"

VanityShouter.new.shout("finished")
```